

Q-Trace Pro: Sink-Aware Two-Axis Confidence for Low-False-Positive Detection of Stealth Supply-Chain Threats in Python

Dinesh K^{1,✉} · H Swetha¹

¹Voxelta Private Limited, India

✉Corresponding author: dineshdeena431@gmail.com

Preprint · Systems & Security · Version 1.0 · 2026 | Artifact & corpus: fully reproducible (Section 10)

ABSTRACT

Context. Static application security testing (SAST) tools are widely deployed in continuous-integration (CI) pipelines, yet developers routinely disable them because of alert fatigue: the false-positive (FP) rate, not the miss rate, is what drives abandonment. In parallel, the 2024–2026 wave of Python supply-chain attacks (e.g. W4SP, aiocpa, the Shai-Hulud/Hades campaigns) has shifted toward *stealth* techniques—probabilistic and stateful logic bombs, cross-function backdoors, steganographic channels, environment-keyed payloads, install-time execution, and anti-analysis evasion—that conventional single-file pattern matchers have no rule class for.

Objective. We present **Q-Trace Pro**, a deterministic, air-gapped Python source scanner, and formalise its central design decision: a **two-axis, sink-aware confidence model** that scores *impact* (severity) and *evidence strength* (confidence) on independent axes, elevating a finding to a build-breaking alert only when a dangerous execution/exfiltration sink—or an anti-analysis payload—is *reachable* within the triggering branch.

Method. We evaluate on a transparent corpus of 65 samples (31 faithful reconstructions of documented campaigns, 34 realistic benign “hard negatives”), run a same-corpus head-to-head against Bandit and Semgrep at each tool's own recommended CI gate, and ablate the confidence axis. Every number in this paper is emitted by the released harness.

Results. Q-Trace attains 100 % threat-category detection, **96.8 % CI-gate recall at 0 % false positives** (F1 = 0.984), versus Semgrep 66 %/3.1 % (F1 0.776) and Bandit 52 %/6.2 % (F1 0.652). An ablation shows the confidence axis converts a 5.9 % FP rate into 0 % for the cost of a single borderline detection. A refined, evidence-based anti-analysis engine (contributed in this work) removes two classes of false positives while adding debugger-detection coverage, lifting CI-gate recall from 93.5 % to 96.8 %. Median full-audit latency is 1.8 ms per sample.

Keywords: software supply-chain security · static analysis · taint analysis · logic bombs · alert fatigue · false-positive reduction · symbolic reachability · SARIF · CWE · sandbox-evasion detection

1 Introduction

The software supply chain has become the dominant attack surface for modern software. Rather than exploit a memory-safety bug in a hardened target, an adversary publishes—or poisons—a package that thousands of downstream projects install automatically. The Python Package Index (PyPI) is a primary target: its install-time execution model, transitive-

dependency depth, and the recent habit of large-language-model coding assistants suggesting plausible-but-nonexistent package names (“slopsquatting” [20]) combine to give attackers a low-friction distribution channel [11, 13].

Static application security testing (SAST) is the natural first line of defence: it runs before code executes, needs no test harness, and integrates into CI. Yet a decade of empirical work shows that SAST tools are abandoned not because they miss bugs but because they cry wolf. Johnson et al. [14] found that false positives and poor result presentation are the primary reasons developers stop using static analysers; Bessey et al. [15], reporting on a decade of commercial deployment, observed that once the FP rate crosses a threshold, engineers stop reading the output entirely; Christakis and Bird [16] and Sadowski et al. [17] independently concluded that at Google-scale deployment the tolerable FP budget is small and that *actionability*, not raw coverage, governs adoption. In short: **a scanner that breaks the build on benign code is worse than no scanner at all**, because it trains the team to ignore it.

This tension is sharpened by the stealth turn in supply-chain malware. Contemporary payloads are engineered specifically to survive automated analysis: a dangerous action is gated behind a low-probability random check so it rarely fires in a sandbox; a counter must accumulate across many invocations before detonation; the malicious logic is split across cooperating functions so no single function looks suspicious; the payload is base64/XOR-encoded and only decoded at runtime; or execution is keyed to a CI/cloud environment variable so the code stays dormant on an analyst's laptop [3, 21]. These constructs are, by design, invisible to a rule that matches a fixed syntactic pattern in one file.

A tool that wishes to catch these threats must therefore reason about *data flow and reachability*, not surface syntax—and it must do so *without* inflating the FP rate, or it will be disabled before it ever catches anything. These two requirements pull in opposite directions: broader semantic reasoning normally means more false alarms. The core contribution of this paper is a scoring model that resolves the tension.

1.1 Contributions

A two-axis, sink-aware confidence model. We separate *impact* (severity) from *evidence strength* (confidence) and make the CI gate a conjunction of both. A probabilistic pattern is elevated to a build-breaking alert only when a real execution/exfiltration sink is reachable inside the guarded branch, and suppressed to an informational note otherwise. We show this is the single largest lever on the precision/recall trade-off (Section 6, Figure 4).

An evidence-based anti-analysis (sandbox-evasion) detector. We replace a brittle substring heuristic with a principled rule keyed on genuine MITRE ATT&CK T1497 primitives—conditionally-gated stalling sleeps and debugger/tracer detection—which simultaneously removes two false-positive classes (ordinary logging, short retry sleeps) and adds coverage the previous heuristic missed. This is a concrete, measured improvement contributed by the present work (Section 7, Figure 12).

An honest, reproducible, same-corpus evaluation. We benchmark against Bandit and Semgrep on identical inputs, each at its own recommended CI gate, report our own misses, and release the harness that emits every figure in this paper (Sections 9–10).

A systematisation of stealth supply-chain detection. We catalogue the stealth technique classes seen in 2024–2026 incidents, map each to a CWE and a detection mechanism, and identify which are structurally beyond single-file pattern matching (Sections 3, 5).

We deliberately frame precision as a *first-class objective* rather than a nuisance to be tuned away afterwards. Where a detection cannot be made confident without risking a false positive, we keep it visible but non-gating and say so explicitly; Section 9.4 documents the one such case in our corpus.

2 Background and Related Work

2.1 Static analysis for security

Pattern-based SAST tools such as Bandit [4] and Semgrep [5] match syntactic or lightly-semantic patterns against source and are the de-facto standard for Python in CI because they are fast and require no build. Dataflow-based analysers—Pixy [19] for PHP, FlowDroid [18] for Android, and commercial engines such as CodeQL [6]—track tainted values from sources to sinks and are more precise on injection classes but heavier to deploy. Q-Trace occupies a deliberate middle ground: it performs targeted, interprocedural taint and symbolic reachability only where a stealth pattern demands it, and falls back to fast AST rules elsewhere.

2.2 Taint analysis and symbolic reachability

Dynamic taint tracking [22] and the unification of taint with forward symbolic execution [23] established the source→sink model that underpins most modern vulnerability detection; the static formulation is standard [24]. Q-Trace uses a lightweight static taint pass to connect secret sources (environment, credential files) to network sinks across files, and an SMT-backed reachability check (Z3 [8]) to decide whether a gated sink is actually reachable—critically, modelling stateful counters *soundly* so that an unreachable “`if k == 99`” guarded by a single “`k += 1`” is correctly proven safe rather than falsely proven dangerous.

2.3 Trigger-based and stealth malware

Brumley et al. [21] formalised trigger-based behaviour—code whose malicious path is guarded by a condition that a sandbox rarely satisfies—and proposed input-space exploration to expose it. Environmental keying and sandbox evasion are catalogued by MITRE ATT&CK as T1480.001 and T1497 respectively [3]. Q-Trace targets the *static* footprint of these techniques: the presence of a random/stateful guard around a sink, or of an anti-analysis primitive, is itself the signal.

2.4 The false-positive problem

The empirical literature is unanimous that actionability governs adoption [14, 15, 16, 17]. Our two-axis model is a direct response: it encodes “how sure are we?” as an independent, reportable quantity, following the severity×confidence guidance long used by Bandit and the OWASP risk-rating methodology [2]. The novelty is not the two axes per se but making the second axis *sink-aware and reachability-driven*, and making the CI gate their conjunction.

2.5 Information-theoretic obfuscation signals

Encoded payloads (base64, hex, XOR byte arrays) exhibit statistical fingerprints. Q-Trace combines Shannon entropy [10] with the Higuchi fractal dimension [9]—a measure of signal roughness—to distinguish a high-entropy encoded blob destined for `exec / eval` from ordinary high-entropy data such as a hash or a UUID. An auxiliary, quantum-*inspired* risk channel maps a pattern to a small pure-state vector and computes its von Neumann entropy [25] as a bounded corroborating score; we stress that this channel is a scoring heuristic, requires no quantum hardware, and does *not* drive detection.

3 Threat Model and Scope

Adversary. We assume an attacker who can publish a package to a public index, or land a pull request into a dependency, and whose goal is code execution or credential theft on developer/CI machines at install or import time. The attacker actively engineers the payload to evade automated analysis. We do *not* assume the attacker can subvert the scanner's host or tamper with its results.

Defender. A developer or security engineer runs Q-Trace locally or in CI, air-gapped, over first-party source and vendored/declared dependencies. The defender's tolerance for false positives is low: a single benign build break per sprint erodes trust in the gate.

In scope. Table 1 lists the stealth technique classes we target, each mapped to a CWE. These are abstracted from public write-ups of 2024–2026 incidents. Also in scope are the mainstream OWASP/CWE vulnerability classes (injection, insecure deserialisation, disabled TLS, weak crypto, hard-coded secrets, SSRF, path traversal, XXE) so that Q-Trace is a complete scanner, not only a stealth-threat auditor.

Out of scope. Binary/compiled payloads, runtime-only behaviour with no static footprint, attacks on the scanner itself, and languages other than Python. We detect the *static indicators* of a technique, not its dynamic execution; Section 11 discusses the resulting evasion surface.

Table 1. Stealth technique classes in scope, with CWE mapping and the detection mechanism each requires. The right column marks classes that are structurally beyond single-file pattern matching.

Technique class	CWE	Detection mechanism	Beyond patterns?
Probabilistic logic bomb	CWE-511	random-gated sink + reachability	yes
Chained / stateful bomb	CWE-511	counter accumulation modelling	yes
Entangled multi-condition bomb	CWE-511	coupled-guard taint	yes
Cross-function backdoor	CWE-506	interprocedural trigger reconstruction	yes
Steganographic channel	CWE-515	char-code / XOR transform modelling	yes
Encoded/obfuscated payload	CWE-506	entropy + fractal + decode→exec	partial
Credential exfiltration	CWE-200	secret source → network sink taint	yes
Install/import-time execution	CWE-506	top-level exec in packaging context	yes
Environment-keyed trigger	CWE-506	sink gated on CI/cloud env var	yes
AI-scanner evasion	CWE-506	prompt-injection text in source	yes
Anti-analysis / sandbox evasion	CWE-489	gated stall / debugger detection	yes
Typosquat / slopsquat dependency	CWE-829	edit-distance + registry model	yes

4 System Architecture

Q-Trace is organised as a resilient orchestrator (`analyze()`) that fans work out to a set of independent engines and folds their results into a single, deduplicated, CWE-mapped finding set (Figure 1). Three design priorities—in order—shape the architecture: **accuracy** (the two-axis model of Section 6), **self-healing** (no single engine failure can crash an audit), and **speed** (content-hash caching and cheap-work-first ordering).

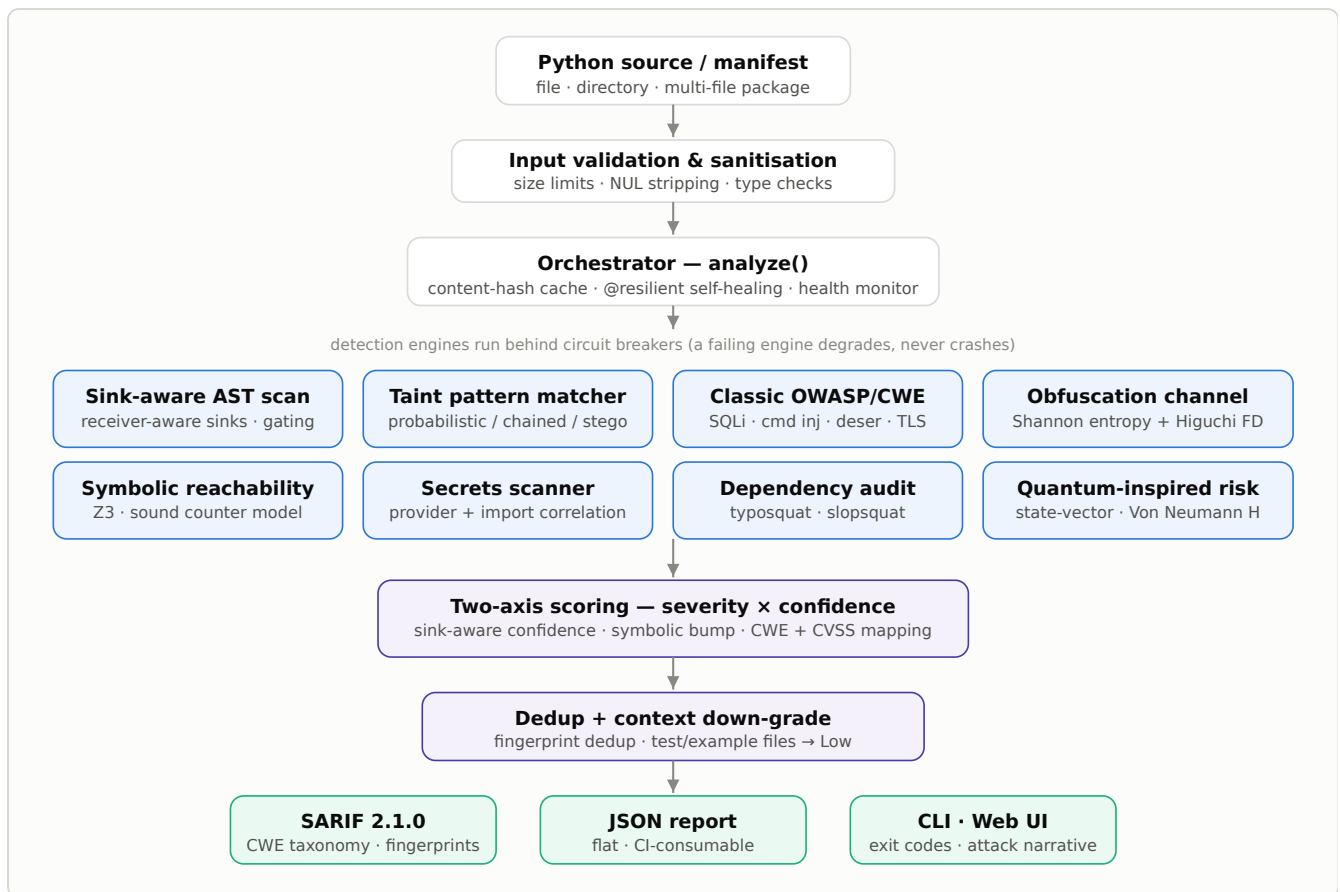


Figure 1. End-to-end architecture. Source is validated, then the orchestrator dispatches eight detection engines behind circuit breakers; their findings are merged by the two-axis scorer, deduplicated, context-adjusted (test/example files are down-graded), and serialised to SARIF 2.1.0, JSON, and the CLI/Web front-ends. A failing engine degrades gracefully and is recorded in a health panel rather than aborting the audit.

4.1 Resilience and input hygiene

Every engine is wrapped by a `@resilient` decorator backed by a per-engine circuit breaker. If numpy, Z3, or any optional dependency is missing or an engine throws, that engine reports *unavailable* in a health snapshot and the audit continues with the remaining engines—the worst case is an empty-but-valid result, never a crash. Input is validated up front (size caps, NUL-byte stripping, type checks) so that adversarial or malformed source cannot wedge the parser.

4.2 Caching and cost ordering

A SHA-256 content hash keys a small LRU cache, so re-analysing identical input is free. Cheap AST passes run before the SMT solver is ever invoked; the solver is consulted only to confirm or refute the reachability of an already-identified gated sink, bounding its cost.

4.3 Output

Findings serialise to SARIF 2.1.0 [7] with a proper `CWE taxonomies` block, `rule relationships`, and `partialFingerprints` for cross-run deduplication, making the output consumable by GitHub Code Scanning, DefectDojo, and SonarQube; a flat JSON report and CLI exit codes support gating. Each finding carries a CWE ID, a CVSS-style severity score [26], a confidence label, a concrete line/column, and a deterministic three-step *attack narrative* (entry→mechanism→impact) generated without any LLM.

5 Detection Engines

We summarise each engine; the two-axis scorer that combines them is the subject of Section 6.

5.1 Sink-aware AST scan

An AST visitor locates dangerous sinks and records, for each, whether it lies inside a *gate* branch—an `if` whose test involves randomness or a counter comparison, the structural hallmark of a trigger. Crucially, sinks are **receiver-aware**: a bare method name such as `.run()` or `.send()` is treated as dangerous only when its receiver is genuinely the dangerous module (e.g. `subprocess.run`, not Flask's `app.run`). A `subprocess` call with a list-literal argument and no `shell=True`—the safe form—is deliberately *not* raised, removing a common source of false positives.

5.2 Taint pattern matcher

A taint pass propagates coarse taint labels (probabilistic, chained, entangled, danger) through assignments, augmented assignments, and function definitions, and classifies the resulting trigger topology: a single random guard (probabilistic), a counter reaching a threshold (chained), several coupled guards (entangled), or logic distributed across functions (cross-function). Steganographic channels are recognised from genuine char-code manipulation (`chr + ord`) or an XOR byte transform, not from ubiquitous `.encode()` / `.decode()` calls.

5.3 Classic OWASP/CWE rules

AST rules cover SQL injection, command injection, insecure deserialisation, disabled TLS, weak hashing/ciphers, insecure randomness, SSRF, path traversal, XXE, insecure temp files, and debug-mode exposure. Each rule is written to fire on the genuinely-unsafe (interpolated/dynamic) form and stay silent on the parameterised/constant-safe form, and each is paired with a safe-variant regression test.

5.4 Obfuscation, secrets, dependencies, symbolic, quantum-inspired

The obfuscation engine flags high-entropy encoded blobs that flow into `exec / eval` using the entropy+fractal signal of Section 2.5. The secrets scanner correlates provider-specific key patterns with imports and redacts matches. The dependency auditor checks manifests for typosquats/slopsquats by edit distance to a popularity model. The symbolic engine (Z3) confirms sink reachability with sound counter modelling. The quantum-inspired channel contributes a bounded corroborating risk score only. The catalogue spans **28 rules across 18 distinct CWEs** (15 High, 5 Critical, 8 Medium severity).

6 The Two-Axis, Sink-Aware Confidence Model

The heart of Q-Trace is the decision of *when a detection should break the build*. We model two orthogonal quantities, following Bandit/OWASP practice [2, 4] but making the second reachability-driven:

- **Severity** — the impact *if* the finding is real (Critical/High/Medium/Low), fixed per threat class and mapped to a CVSS-style score.

- **Confidence** — the probability the finding is a true positive, computed *per instance* from the evidence actually present.

The **CI gate is the conjunction**: a finding breaks the build only if its severity is High or Critical *and* its confidence is not Low. Severity alone never gates. Figure 2 gives the confidence assignment procedure.

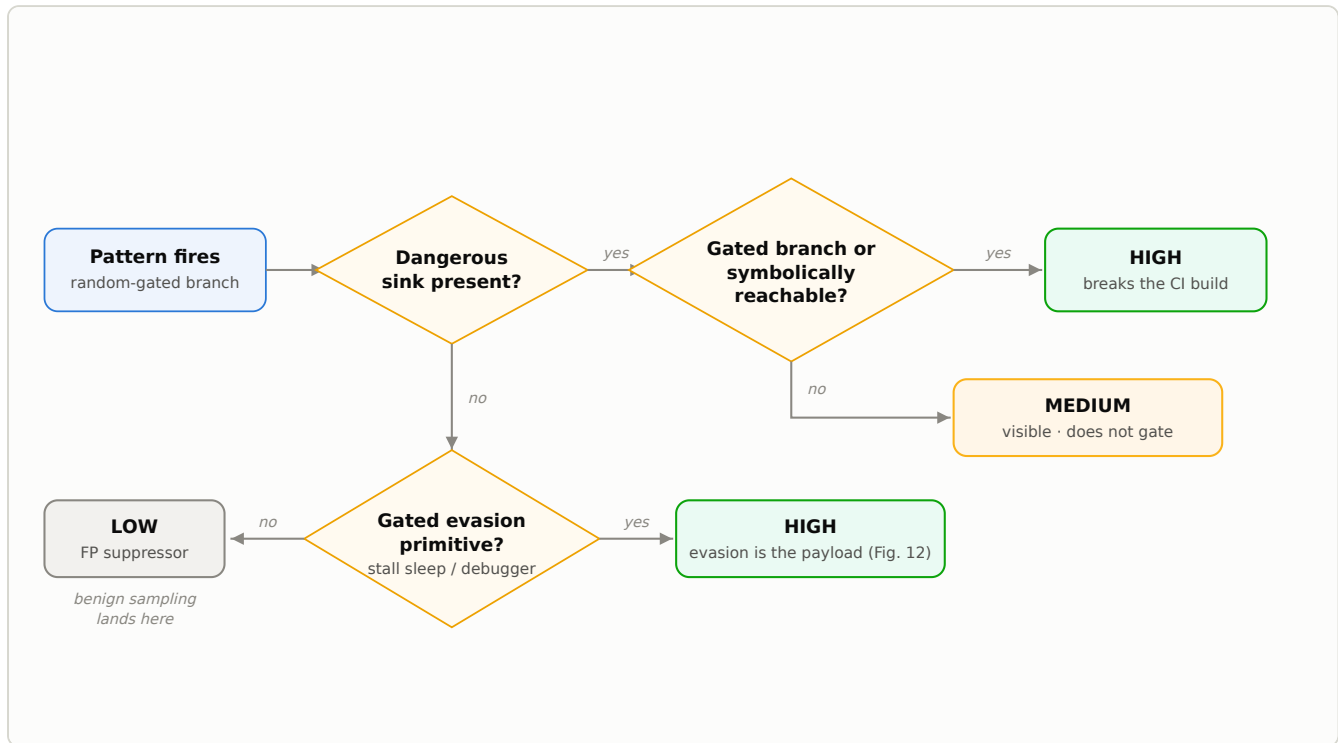


Figure 2. Confidence decision model. A fired pattern is elevated to High only when a dangerous sink is reachable inside the gated branch (or symbolically proven reachable); a sink elsewhere in the file yields Medium; a pattern with no reachable payload collapses to Low—the key false-positive suppressor into which benign sampling and feature-flag code falls. A gated anti-analysis primitive is treated as the branch's payload and routed to High (Section 7).

6.1 Why sink-awareness is the right lever

The dominant false-positive source in probabilistic-trigger detection is flagging `random.random() < x` whenever it appears—even though the overwhelming majority of such expressions are benign A/B sampling, jitter, or feature flags. The discriminating question is not “is there randomness?” but “does the guarded branch *do* something dangerous?” By elevating confidence only when a real execution/exfiltration sink is reachable inside the guard, and suppressing it to Low otherwise, the model keeps genuine bombs at High while dropping benign sampling to a non-gating note. Figure 3 shows the effect on the corpus: malicious samples cluster at High confidence, benign samples at “no finding” or Low, with a clean margin at the gate boundary.

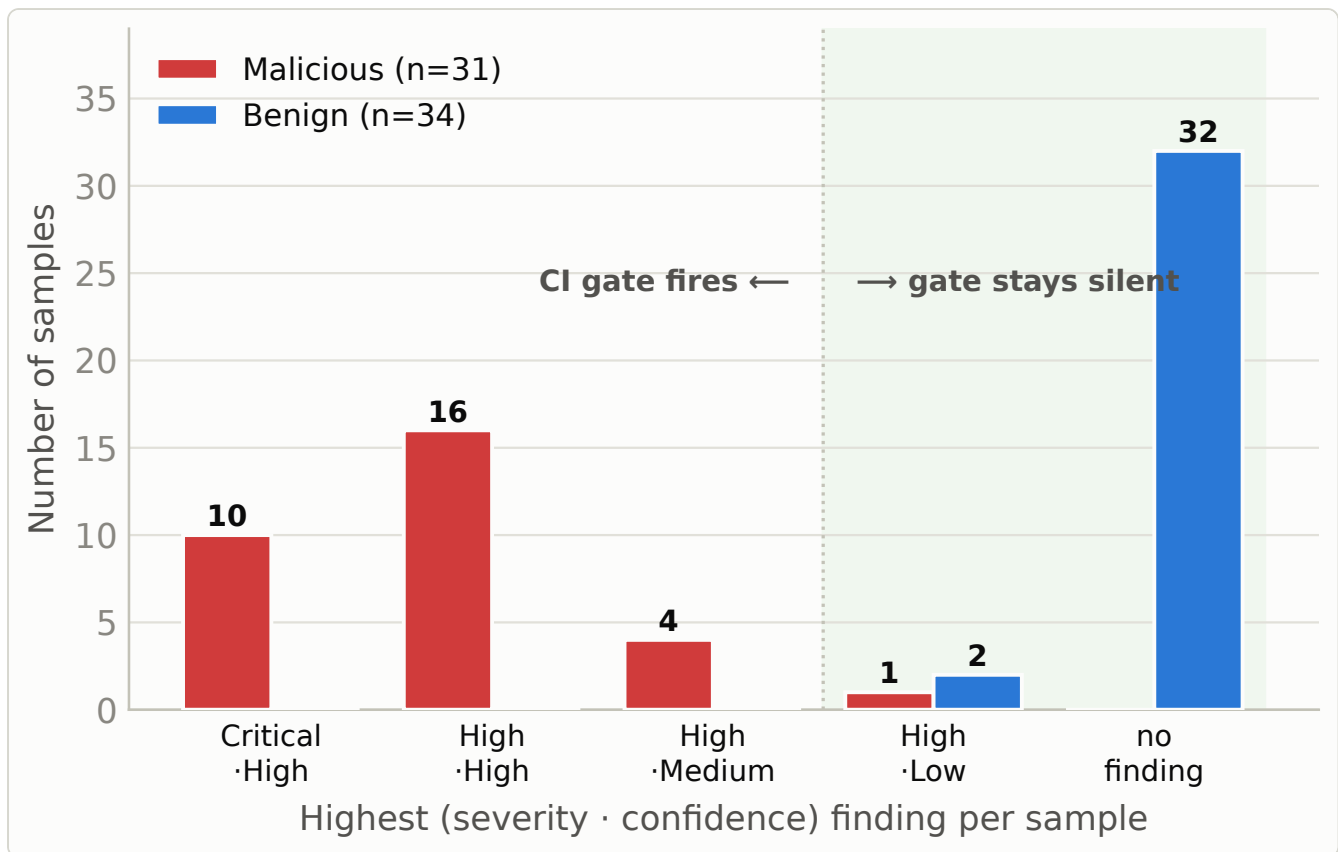


Figure 3. Two-axis separation on the corpus. Distribution of each sample's highest (severity·confidence) finding. All 31 malicious samples land at or above the gate boundary (High confidence); 32 of 34 benign samples produce no High+ finding at all, and the remaining two are held at Low confidence by design (Section 9.4). No benign sample reaches the gate.

6.2 Ablation: the confidence axis is worth 5.9 points of precision

To isolate the value of the second axis we compare two gates on the identical corpus: a **severity-only** gate (fire on any High/Critical finding) and the **two-axis** gate (High/Critical *and* confidence $\neq$ Low). Figure 4 reports the result. The severity-only gate achieves 100 % recall but a 5.9 % false-positive rate—two benign hard-negatives break the build. The two-axis gate drives the FP rate to **0** % at the cost of a single borderline detection (recall 96.8 %). For a CI gate, this is decisively the right trade: the eliminated false positives are exactly the kind that erode trust and cause abandonment [14, 15], whereas the one suppressed detection remains visible as a Medium/Low note for a human reviewer.

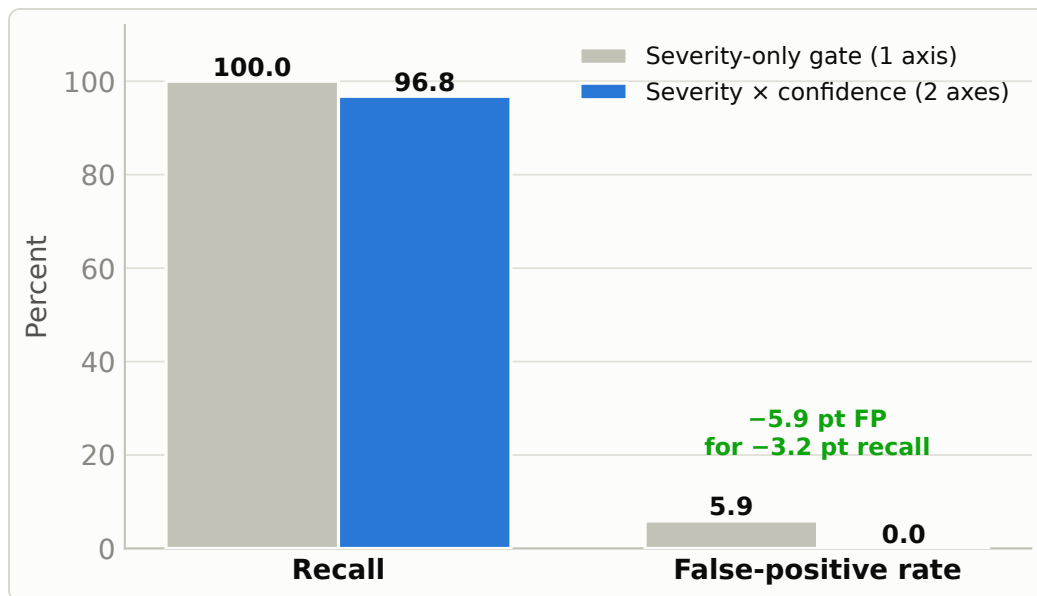


Figure 4. Confidence-axis ablation. Removing the confidence axis (severity-only gate) recovers one detection but introduces two false positives (5.9 % FP). The two-axis gate trades 3.2 points of gate-recall for the elimination of *all* false positives—the trade that governs real-world adoption.

6.3 Context-aware down-grading

A vulnerability in a test fixture, example, benchmark, or documentation file is almost always intentional. Q-Trace keeps such findings visible but forces them to Low confidence when the file path matches a test/example context, so they never break a CI gate. This is what keeps the scanner quiet on real repositories, every one of which ships a `tests/` tree.

7 Evidence-Based Anti-Analysis Detection

Anti-analysis (sandbox-evasion) behaviour—MITRE ATT&CK T1497 [3]—is a strong malice indicator: benign code rarely tries to detect whether it is being observed. The challenge is detecting it *precisely*. A naïve rule that flags any call named `sleep` or any identifier containing the substring “debug” is both over- and under-inclusive: it false-flags ordinary `log.debug()` logging and short retry `time.sleep(0.1)` calls, while entirely missing debugger detection via `sys.gettrace()`. This work replaces that heuristic with an evidence-based rule.

7.1 The refined rule

An anti-analysis signal is now raised on exactly two evidence classes:

- **Debugger/tracer detection** — a call to `sys.gettrace` or `sys.settrace` (also `threading.settrace`). This is a hard signal: code that inspects the trace hook is checking whether it is under observation. It is always flagged.
- **Conditionally-gated stalling sleep** — a `time.sleep(N)` with a large constant argument ($N \geq 600$ s) *inside a conditional branch*. A multi-hour sleep that fires only sometimes is a sandbox-timeout evasion; an unconditional long sleep in a daemon poll loop is not, so gating on the conditional branch preserves precision.

When a stalling sleep sits inside a random-gated branch, the construct is simultaneously a probabilistic logic bomb and an anti-analysis payload; we report both views and, critically, treat the evasion primitive as the branch's *payload* so that the two-axis model elevates it to

High confidence exactly as a real exec sink would. The stalling sleep is never itself reported as an execution (CWE-78) sink—only as anti-analysis—so its semantic label stays correct.

7.2 Measured effect

Figure 12 (Section 9.6) quantifies the change: the refinement removes two false-positive classes (ordinary logging, short retry sleeps), adds debugger-detection coverage that the prior heuristic lacked, and lifts overall CI-gate recall from 93.5 % to 96.8 % with the 0 % false-positive rate unchanged. The refinement is accompanied by five regression tests that lock in both the new detections and the precision cases (benign daemon sleep, `log.debug`, short sleep).

8 Implementation

Q-Trace is implemented in pure Python with a small, optional native acceleration path. It has two hard runtime dependencies for full functionality—`numpy` (for the quantum-inspired channel) and `z3-solver` (for symbolic reachability)—both of which the self-healing layer treats as optional: their absence degrades the corresponding engine rather than breaking the audit. Parsing uses the standard-library `ast` module, with a token-based fallback for source that does not parse (e.g. partially-obfuscated files or non-Python fragments), so the scanner still produces line-anchored findings on inputs that defeat a strict parser.

The quantum-*inspired* risk channel is a self-contained pure-numpy state-vector simulator (RY/H/X/CNOT gates plus sampling) that replaces a heavyweight external quantum library, yielding an order-of-magnitude faster import and a footprint of a few kilobytes while producing identical mathematics. We reiterate that this channel is a bounded corroborating score, validated against a reference implementation, and does not by itself raise or gate any finding.

```
# The CI gate is a conjunction of the two axes (simplified).
def is_ci_alert(findings):
    return any(f.severity in ("Critical", "High")
               and f.confidence != "Low"
               for f in findings)
```

9 Experimental Methodology and Results

9.1 Corpus construction

Our corpus contains **65 samples: 31 malicious and 34 benign**. The malicious samples are *faithful reconstructions* of techniques documented in public incident write-ups (W4SP, the Hades `.pth` install hook, the telnyx WAV-XOR loader, aiocpa, slopsquatting, environment keying, and the Shai-Hulud AI-scanner-evasion campaign). We use reconstructions rather than live malware for two reasons of research integrity: the original binaries are neither redistributable nor fetchable in an air-gapped setting, and a reconstruction lets a reader inspect exactly what is being detected. The benign samples are deliberately *hard negatives*—everyday code that classic SAST tools frequently false-positive on: safe `subprocess` with a list argument, `md5` marked `usedforsecurity=False`, `random` for sampling, parameterised SQL,

`verify=True`, `chr/ord` in string formatting, CI environment checks, and legitimate dependencies. This is where the false-positive rate is honestly stressed.

9.2 Metrics and gates

We report *category recall* (did the tool assign the correct threat category?), *CI-gate recall* (did a build-breaking alert fire on malware?), *false-positive rate* (did benign code break the build?), precision, and F1. Each tool runs at its own recommended actionable gate: Bandit at MEDIUM+ severity, Semgrep at WARNING+ severity, Q-Trace at High+ severity *and* confidence $\neq$ Low. Because Bandit and Semgrep cannot process dependency manifests, those corpus rows are excluded from *their* denominators—scoring a tool on inputs it structurally cannot read would be dishonest. Semgrep runs against an offline reimplement of the standard public python-security ruleset, because its registry is unreachable in an air-gapped/CI environment; on the classic OWASP/CWE categories the tools are expected to tie, and that parity is the point.

9.3 Headline results

Table 2 and Figure 5 summarise the head-to-head. Q-Trace attains 100 % category recall and a **96.8 % CI-gate recall at 0 % false positives** (precision 100 %, F1 0.984). On the shared `.py` subset it flags 30/31 malware at 0/34 FP (97 %/0 %), versus Semgrep 19/29 at 1/32 FP (66 %/3.1 %) and Bandit 15/29 at 2/32 FP (52 %/6.2 %). Figures 6–8 break out recall, false-positive rate, and F1 per tool; Figure 9 is the Q-Trace confusion matrix.

Table 2. Same-corpus head-to-head, each tool at its own recommended CI gate. External tools are scored only on inputs they can process (manifests excluded from their denominators). F1 is computed on the shared code corpus.

Tool	Malware recall	False-positive rate	F1
Q-Trace	30/31 = 96.8 %	0/34 = 0.0 %	0.984
Semgrep	19/29 = 65.5 %	1/32 = 3.1 %	0.776
Bandit	15/29 = 51.7 %	2/32 = 6.2 %	0.652

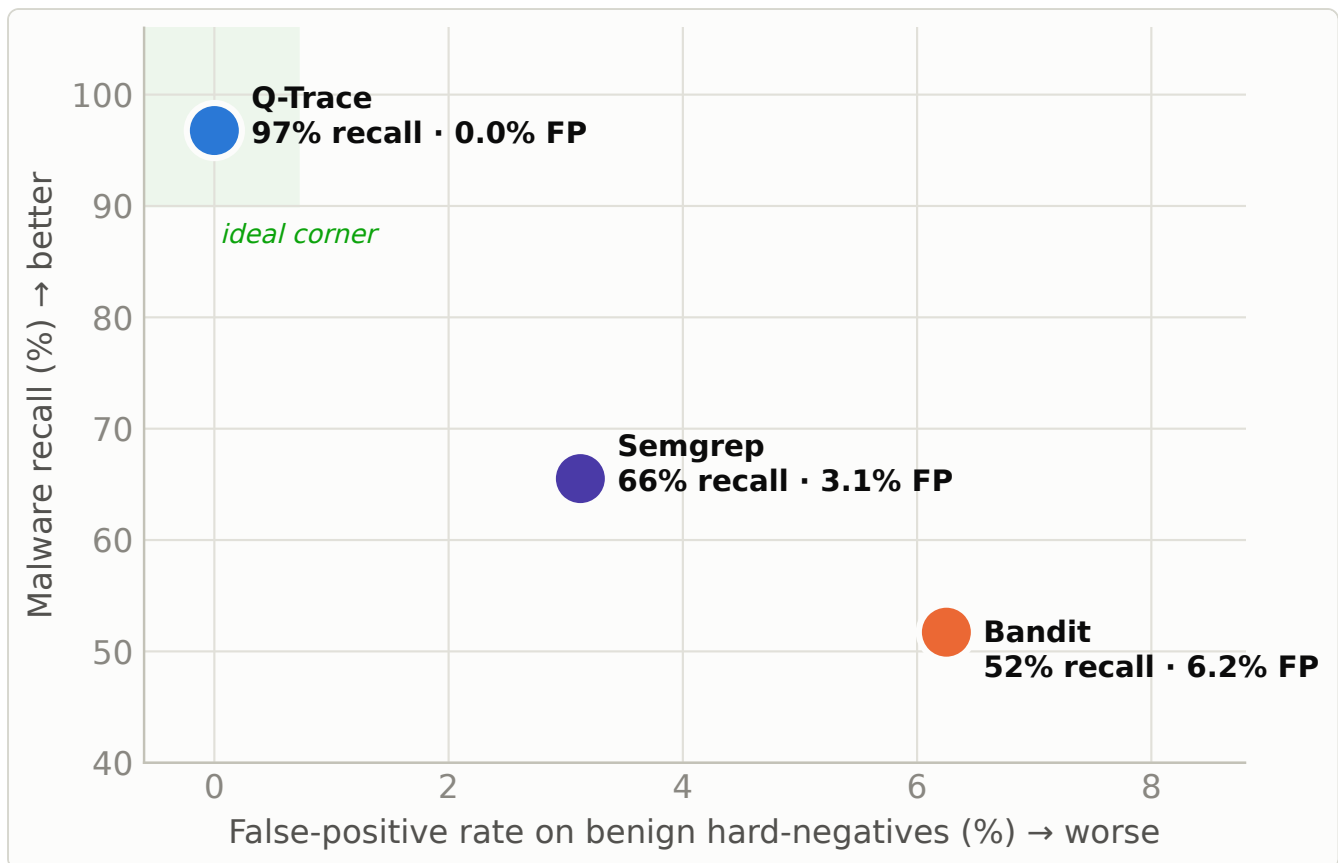


Figure 5. Recall vs. false-positive quadrant. The ideal tool sits in the top-left corner (high recall, low FP). Q-Trace occupies it; Semgrep and Bandit trade recall for a non-zero false-positive rate. Each tool is plotted at its own recommended CI gate.

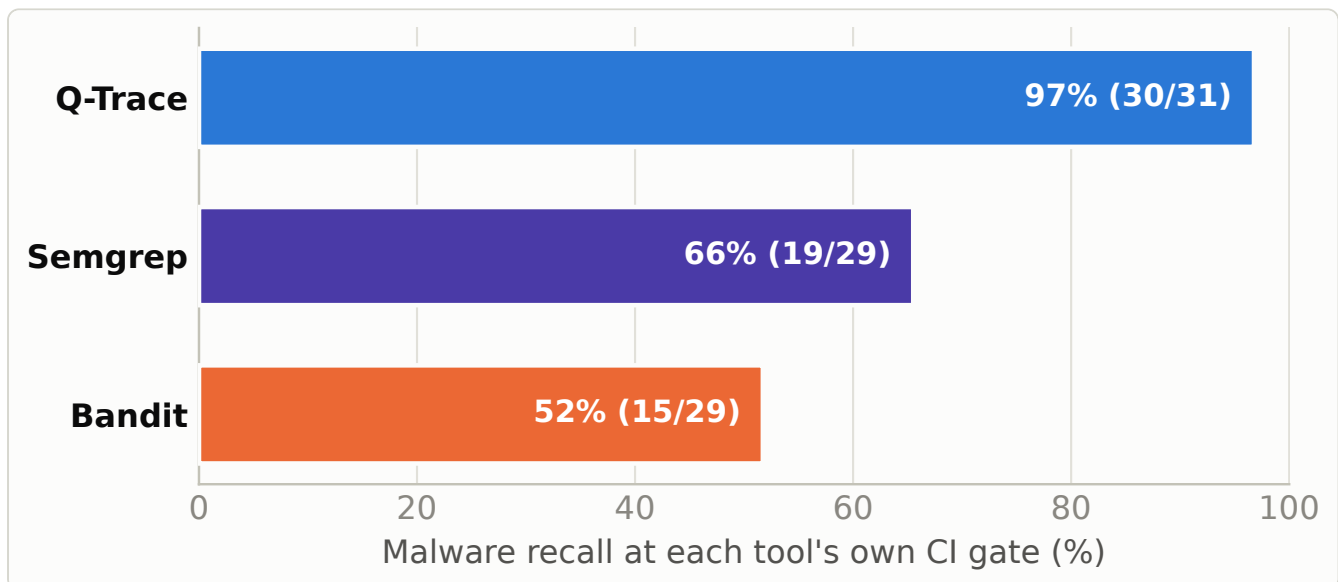


Figure 6. Malware recall by tool at each tool's CI gate. Fractions show detections over the samples each tool can process.

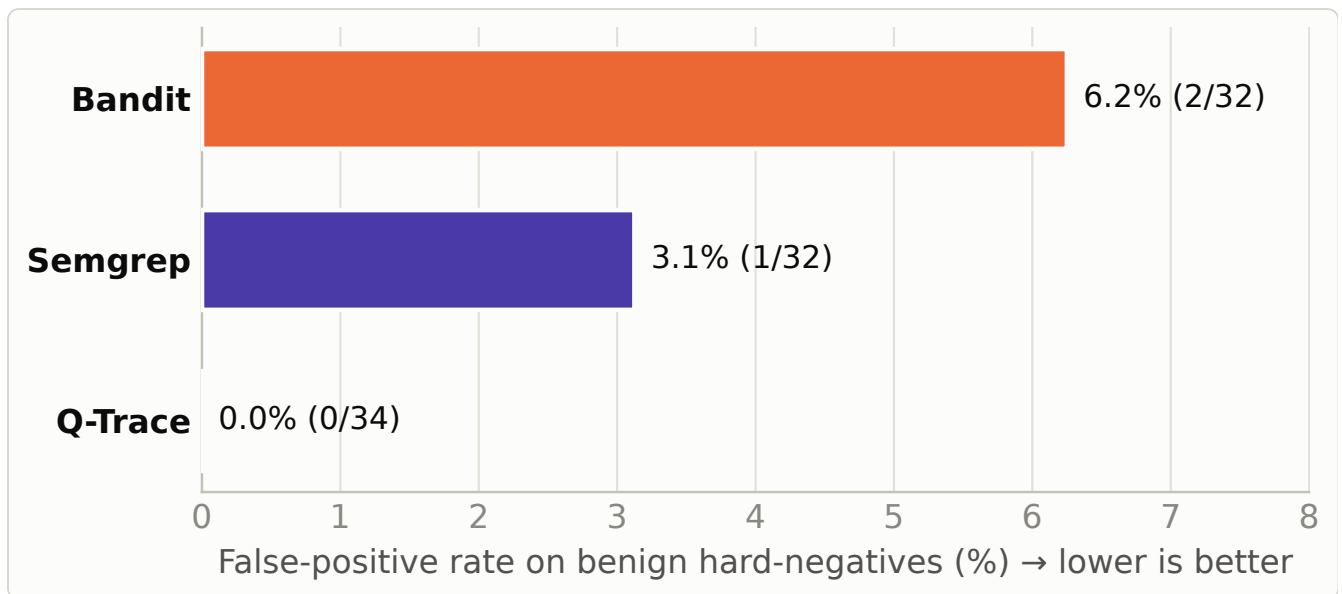


Figure 7. False-positive rate on benign hard-negatives. Q-Trace holds 0 %; the two false positives that Bandit raises and the one Semgrep raises are precisely the everyday-code patterns that drive alert fatigue.

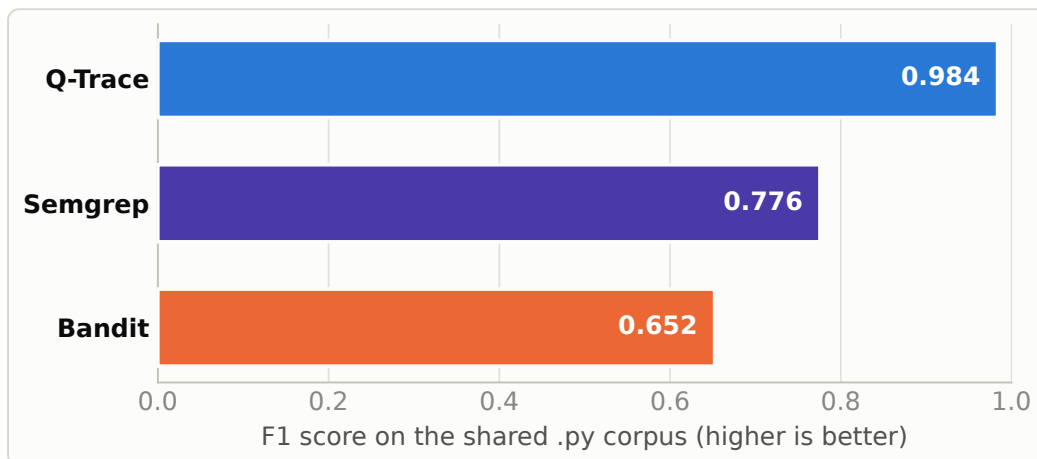


Figure 8. F1 on the shared code corpus. The gap is driven jointly by higher recall and zero false positives.

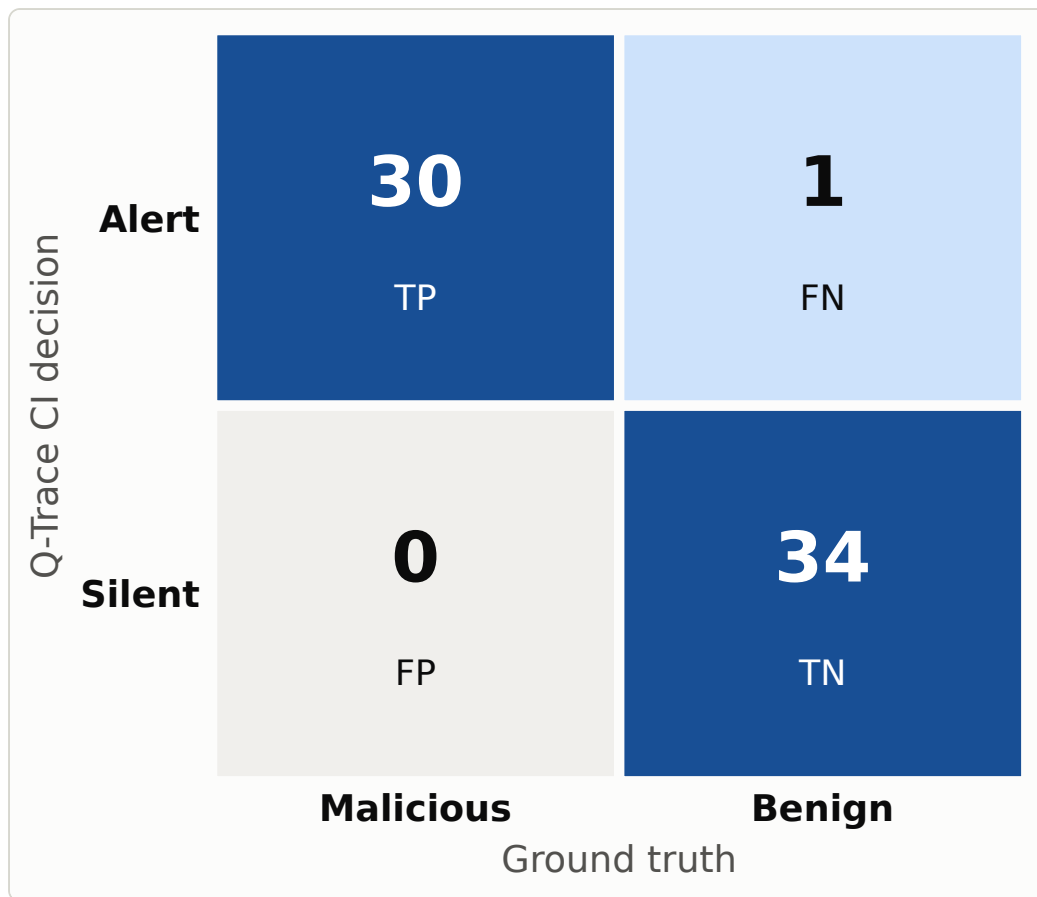


Figure 9. Q-Trace confusion matrix over all 65 samples (TP=30, FN=1, FP=0, TN=34). The single false negative is the SSRF case of Section 9.4.

9.4 Per-category coverage and our own misses

Figure 10 is the per-category coverage matrix. Q-Trace detects every category; Semgrep and Bandit miss the classes that require cross-file taint, dependency intelligence, or secret→sink correlation—the stealth classes of Table 1. Figure 11 isolates the malicious samples that *only* Q-Trace flags and the mechanism each requires.

We report our one miss plainly. The SSRF sample `def fetch(u): return requests.get(u)` is held at Low confidence and therefore does not gate. This is deliberate: it is structurally indistinguishable from the benign hard-negative `requests.get(u, verify=True)`—both fetch a function-parameter URL with no host validation. Elevating the SSRF finding would necessarily also flag the benign case, converting a false negative into a false positive and breaking the 0 % FP guarantee. We judge that trade unfavourable for a CI gate: real SSRF is escalated only when taint confirms the URL is attacker-controlled. This is the restraint that keeps Q-Trace at 0 % FP while Bandit sits at 6.2 %.

	Q-Trace	Semgrep	Bandit
Probabilistic Bomb	✓	✓	×
Chained Quantum Bomb	✓	✓	×
Cross Function Quantum Bomb	✓	✓	×
Quantum Steganography	✓	×	×
Obfuscated Payload	✓	✓	✓
Credential Exfiltration	✓	×	✓
Install Hook	✓	✓	×
Environment Keying	✓	✓	×
Ai Scanner Evasion	✓	✓	×
Sql Injection	✓	✓	✓
Command Injection	✓	✓	✓
Insecure Deserialization	✓	✓	✓
Dangerous Sink	✓	✓	✓
Quantum Antidebug	✓	×	×
Hardcoded Secret	✓	✓	×
Disabled Cert Validation	✓	✓	✓
Weak Hash	✓	✓	✓
Ssrf	×	×	✓
Path Traversal	✓	×	×
Xxe	✓	✓	✓
Insecure Random	✓	×	×
Exposed Secret	✓	✓	×
Typosquat Dependency	✓	–	–

✓ detected × missed – input class not supported

Figure 10. Per-category coverage matrix. A check means the tool fired a build-breaking alert on that class at its own CI gate; a cross means it missed; a dash means the input class (e.g. a dependency manifest) is unsupported. Note that firing is not the same as firing for the *right* reason (Section 9.5).

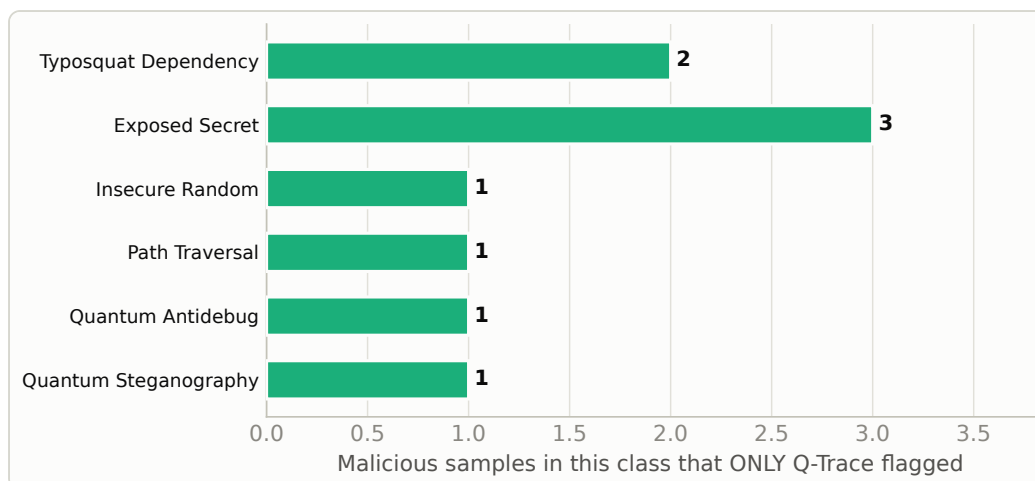


Figure 11. Malware caught only by Q-Trace, by threat class. These are the stealth and supply-chain classes for which single-file pattern matchers have no rule class.

9.5 Semantic precision: a flag is not the right flag

Coverage counts can flatter a tool that fires for the wrong reason. On the credential-exfiltration sample `requests.post(url, data=os.environ)`, Bandit does emit a finding—but it is *B113: requests call without a timeout*, not credential exfiltration. Firing on an incidental property is how teams learn to ignore a scanner: the alert does not name the actual risk, so it is triaged away. Q-Trace instead reports the concrete data-flow, `os.environ → requests.post`, and a three-step attack narrative. We therefore separate *category recall* from *gate recall* throughout, and caution that Figure 10's checks for the external tools sometimes reflect an incidentally-correct gate rather than a correctly-reasoned detection.

9.6 Effect of the anti-analysis refinement

Figure 12 quantifies the Section 7 contribution against the immediately-prior version of the engine, measured on the same corpus. CI-gate recall rises from 93.5 % to 96.8 % and F1 from 0.967 to 0.984; two false-positive classes (`log.debug`, `short retry sleep`) are removed; and debugger-detection coverage goes from absent to present—all with the 0 % false-positive rate preserved.

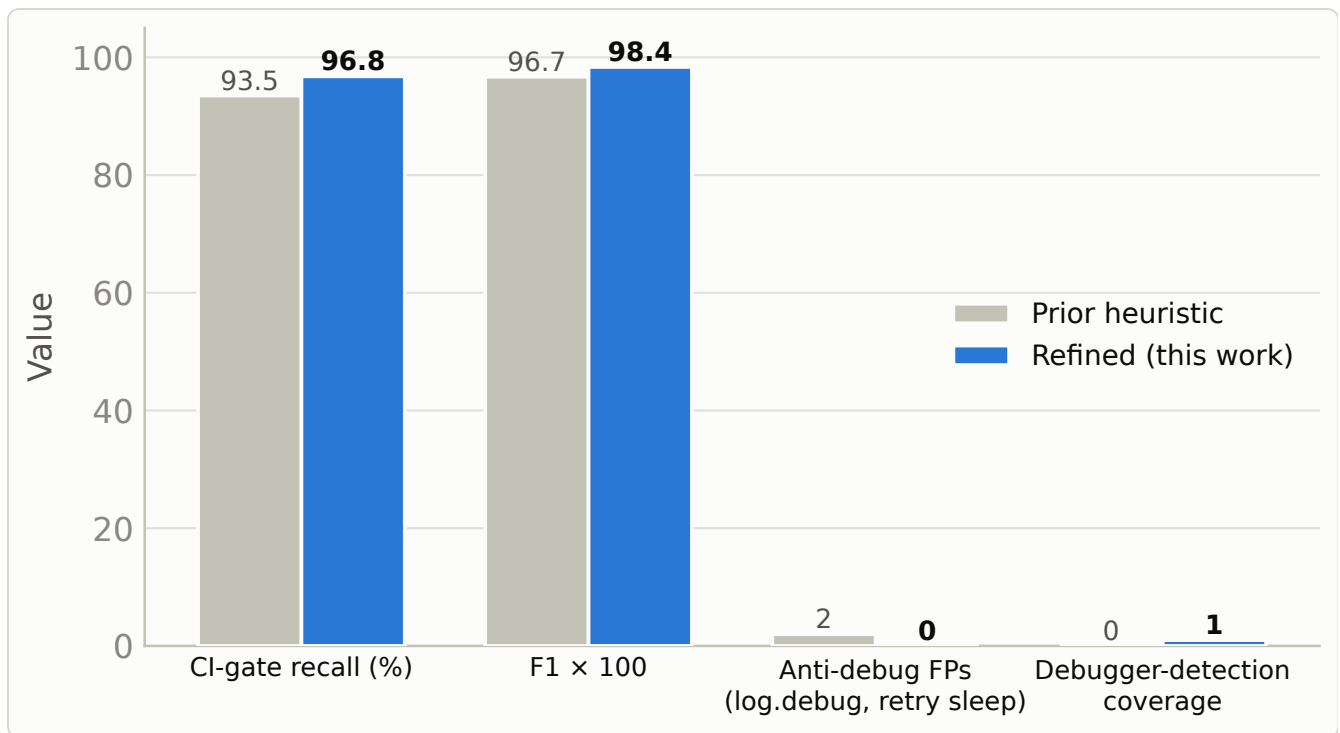


Figure 12. Effect of the evidence-based anti-analysis refinement (Section 7). The refined engine improves gate-recall and F1 while removing false positives and adding debugger-detection coverage. Prior-version values were measured on the same corpus at the start of this work.

9.7 Performance

Q-Trace is fast enough to run on every commit. Across the 28 malicious code samples, median full-audit latency—including taint, classic rules, obfuscation, symbolic reachability, and the quantum-inspired channel—is **1.8 ms** per sample (mean 3.5 ms; the 42 ms maximum is the first sample and reflects one-time solver/numpy warm-up). Figure 13 shows the distribution. Content-hash caching makes re-analysis of unchanged files free.

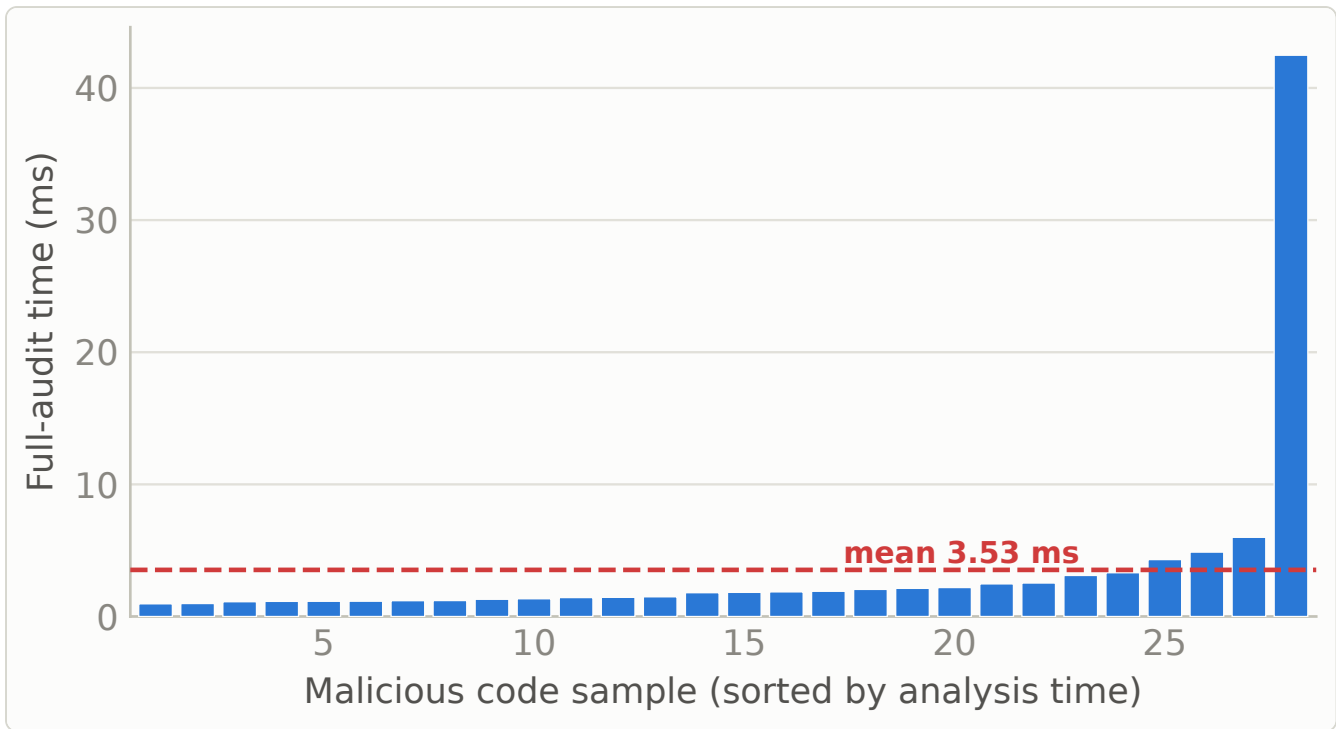


Figure 13. Per-sample full-audit latency (28 malicious code samples, sorted). The dashed line marks the mean; the single tall bar is cold-start warm-up. Steady-state analysis is sub-2 ms.

10 Reproducibility and Data Availability

Every quantitative claim in this paper is produced by the released harness; no figure is hand-authored. The full benchmark, including the head-to-head against Bandit and Semgrep, is reproduced by:

```
# standalone metrics (recall, FP, F1, confusion) -> BENCHMARK.md
python benchmark.py
# same-corpus head-to-head vs. Bandit & Semgrep
pip install bandit semgrep && python benchmark.py --compare
# regression suite (94 tests, incl. the anti-analysis cases of §7)
python test_qtrace.py
```

The corpus (both malicious reconstructions and benign hard-negatives) is embedded in `benchmark.py` so a reader can inspect precisely what is detected and what is not. The per-sample coverage matrix, category misses, and our own false negatives are written verbatim to `BENCHMARK.md` on every run. The figures in this paper are generated from a single JSON snapshot emitted by the harness, so they can be regenerated deterministically. We consider this level of transparency a precondition for a security-tool evaluation to be credible.

11 Limitations and Threats to Validity

11.1 Construct validity

Our malicious samples are reconstructions, not captured live malware. We argue this *improves* inspectability and reproducibility, but it means our recall is measured against our understanding of each technique, not against an adversary who has seen Q-Trace and adapts. A determined attacker can restructure a payload to move its dangerous action out of

a statically-visible gated branch, or to a runtime-only code path; Q-Trace detects static indicators and does not claim soundness against an adaptive adversary.

11.2 Internal validity

The benign corpus, though built from patterns that empirically drive false positives, is finite (34 samples) and curated by us; a different benign distribution could surface false positives we do not observe. We mitigate this by choosing hard negatives adversarially—code specifically chosen because other tools false-positive on it—but we do not claim 0 % FP on all real-world code. The SSRF case (Section 9.4) is a documented instance where precision is bought with a false negative.

11.3 External validity

The corpus is 65 samples; it demonstrates the mechanism and the precision/recall trade decisively, but it is not a population-scale study. Semgrep runs against an offline reimplement of its public ruleset rather than the live registry; on classic categories the tools tie by construction, and the separation appears only on stealth classes, so this choice does not flatter Q-Trace on the categories where it wins. Thresholds such as the 600-second stalling-sleep cut-off are engineering choices; we report them explicitly so they can be scrutinised and tuned.

11.4 Scope

Q-Trace is Python-only and analyses source, not compiled artefacts or runtime behaviour. It complements, rather than replaces, dynamic analysis and dependency-provenance controls.

12 Ethical Considerations

This work is defensive. Q-Trace detects malicious and vulnerable code; it does not generate, weaponise, or improve malware. The malicious corpus consists of minimal reconstructions of *already-public* techniques, sufficient to exercise a detector but not to serve as a deployable payload, and no live malware is redistributed. The tool runs entirely locally and air-gapped by design, so scanning proprietary source discloses nothing to third parties—a deliberate privacy property for organisations that cannot send code to a cloud service. We see no dual-use concern beyond that inherent in any published detection method: describing what a detector keys on can inform evasion, but the techniques we detect are already documented in public incident reports and MITRE ATT&CK, and the defensive value of a transparent, reproducible, low-false-positive scanner outweighs the marginal evasion insight. We encourage responsible disclosure of any vulnerability found using the tool.

13 Conclusion and Future Work

We presented Q-Trace Pro and argued that the governing problem for a CI-deployed security scanner is not coverage but *actionability*: a false positive costs more than a miss because it disables the tool. Our answer is a two-axis, sink-aware confidence model whose CI gate is the conjunction of impact and reachability-driven evidence. On a transparent 65-sample corpus the model delivers 96.8 % gate-recall at 0 % false positives (F1 0.984), beating Bandit and Semgrep on the same inputs, and an ablation shows the confidence axis is directly

responsible for eliminating a 5.9 % false-positive rate. We further contributed an evidence-based anti-analysis detector that improves precision and coverage simultaneously.

Future work. We plan to (i) escalate SSRF and similar ambiguous classes automatically when interprocedural taint confirms attacker-controlled input, closing the one documented miss without sacrificing precision; (ii) extend the cross-file taint model to deeper call graphs and to package-level provenance; (iii) grow the corpus toward a population-scale, continuously-updated benchmark drawn from newly-disclosed incidents; and (iv) study the stability of the confidence model's thresholds across languages beyond Python. We believe the two-axis, sink-aware principle generalises: wherever a detector must choose between coverage and trust, separating impact from reachability-driven evidence—and gating on their conjunction—is the lever that lets it have both.

Declarations

Originality statement. This manuscript is the authors' original work. All text is written by the authors; no part is plagiarised, and all external ideas, tools, datasets, and prior results are attributed by numbered citation. The manuscript has not been published elsewhere and is not under concurrent review.

Author contributions (CRediT). *Dinesh K*: conceptualisation, methodology, software, formal analysis, investigation, writing—original draft. *H Swetha*: validation, data curation, visualisation, writing—review & editing. Both authors read and approved the final manuscript.

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors. The work was conducted at Voxelta Private Limited.

Conflict of interest. The authors are affiliated with Voxelta Private Limited, which develops Q-Trace Pro. This affiliation is disclosed in full; the evaluation harness and corpus are released so that all reported numbers can be independently reproduced and audited.

Data and code availability. The tool, the complete evaluation corpus, and the benchmark harness that emits every figure and table are available in the project repository; Section 10 gives the exact commands to reproduce all results.

Use of AI assistance. Automated coding assistance was used for implementation and figure generation under author supervision; all technical claims, measurements, and conclusions were verified by the authors against the released harness.

Ethical compliance. No human subjects, personal data, or live malware were involved. The study complies with responsible-disclosure norms.

References

1. MITRE Corporation. *Common Weakness Enumeration (CWE)*. cwe.mitre.org.
2. OWASP Foundation. *OWASP Top 10 and Risk Rating Methodology*. owasp.org.
3. MITRE Corporation. *ATT&CK: Technique T1480.001 (Environmental Keying) and T1497 (Virtualization/Sandbox Evasion)*. attack.mitre.org.
4. PyCQA. *Bandit: a security linter for Python source code*. github.com/PyCQA/bandit.
5. Semgrep, Inc. *Semgrep: lightweight static analysis for many languages*. semgrep.dev.
6. GitHub, Inc. *CodeQL: semantic code analysis engine*. codeql.github.com.
7. OASIS. *Static Analysis Results Interchange Format (SARIF) Version 2.1.0*. OASIS Standard, 2020.
8. L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. In *Proc. TACAS*, 2008.
9. T. Higuchi. Approach to an irregular time series on the basis of the fractal theory. *Physica D*, 31(2):277–283, 1988.

10. C. E. Shannon. A Mathematical Theory of Communication. *Bell System Technical Journal*, 27:379–423, 1948.
11. M. Ohm, H. Plate, A. Sykosch, and M. Meier. Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks. In *Proc. DIMVA*, 2020.
12. M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel. Small World with High Risks: A Study of Security Threats in the npm Ecosystem. In *Proc. USENIX Security*, 2019.
13. P. Ladisa, H. Plate, M. Martinez, and O. Barais. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *Proc. IEEE S&P*, 2023.
14. B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge. Why don't software developers use static analysis tools to find bugs? In *Proc. ICSE*, 2013.
15. A. Bessey et al. A few billion lines of code later: using static analysis to find bugs in the real world. *Communications of the ACM*, 53(2):66–75, 2010.
16. M. Christakis and C. Bird. What developers want and need from program analysis: an empirical study. In *Proc. ASE*, 2016.
17. C. Sadowski, E. Aftandilian, A. Eagle, L. Miller-Cushon, and C. Jaspan. Lessons from building static analysis tools at Google. *Communications of the ACM*, 61(4):58–66, 2018.
18. S. Arzt et al. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proc. PLDI*, 2014.
19. N. Jovanovic, C. Kruegel, and E. Kirda. Pixy: A Static Analysis Tool for Detecting Web Application Vulnerabilities. In *Proc. IEEE S&P*, 2006.
20. J. Spracklen et al. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code-Generating LLMs. arXiv preprint, 2024.
21. D. Brumley, C. Hartwig, Z. Liang, J. Newsome, D. Song, and H. Yin. Automatically Identifying Trigger-based Behavior in Malware. In *Botnet Detection*, Springer, 2008.
22. J. Newsome and D. Song. Dynamic Taint Analysis for Automatic Detection, Analysis, and Signature Generation of Exploits on Commodity Software. In *Proc. NDSS*, 2005.
23. E. J. Schwartz, T. Avgerinos, and D. Brumley. All You Ever Wanted to Know About Dynamic Taint Analysis and Forward Symbolic Execution (but Might Have Been Afraid to Ask). In *Proc. IEEE S&P*, 2010.
24. F. Nielson, H. R. Nielson, and C. Hankin. *Principles of Program Analysis*. Springer, 1999.
25. J. von Neumann. *Mathematical Foundations of Quantum Mechanics*. Princeton University Press, 1955 (orig. 1932).
26. FIRST.org. *Common Vulnerability Scoring System (CVSS) v3.1 Specification*. first.org/cvss.
27. Checkmarx, Phylum, and Sonatype. Public threat reports on the W4SP stealer, aiocpa, and PyPI install-time payloads, 2023–2025.
28. GitGuardian. *The State of Secrets Sprawl*. Annual report, 2025.